CS1910 Computer Science I • Problem Solving and Debugging **Partner workbook**

Basketball, Revisited

In lecture with a partner • about 25 minutes

Name: _____ **Partner:** _____

This PDF can be filled in: click in an answer space to type, or print it and write. Draw diagrams with your PDF viewer's pen or markup tool, or on paper.

What this workbook practises

You have seen the one-on-one basketball problem before: a game goes to 11, you must win by 2, and the record is a string like `"A2B2A2A2A1A1A1A2"` (player, then points). This time you will slow down and use the problem-solving process deliberately, then use `print` to find a bug in a solution that *looks* right. (The *Ants on a Stick* and *Bit Parity* problems from this lecture have their own workbooks.)

1 Understand and design

about 10 minutes

1. What are we being asked? What is the input, and what is the output?

2. What is redundant: what information in the record do we *not* need to keep once we have used it?

3. Most students designed a solution with variables counting each player's score along the whole game. Can you decompose the problem into smaller parts?

4. Build examples other than the one given. Include at least one *normal* case and one *edge* case, with the expected winner.

2 Debug by printing

about 12 minutes

A classmate wrote this solution. It passes the two normal cases, but on the deuce case it says A wins, when B should.

```
def get_winner(scoring_sequence):
    score_a = 0
    score_b = 0
    index = 0
    while index < len(scoring_sequence):
        current_player = scoring_sequence[index]
        points_scored = int(scoring_sequence[index + 1])
        if current_player == "A":
            score_a = score_a + points_scored
        else:
            score_b = score_b + points_scored
        if score_a >= 11 or score_b >= 11:
            if score_a > score_b:
                return "A"
            else:
                return "B"
        index = index + 2
    return "A" if score_a > score_b else "B"
```

5. Where would you add a `print` to watch what happens? What would you print?

6. Run (or trace) the deuce case with your `print`. Fill in the last few lines of output.

Basket	Score A	Score B
...	...	...
B1 (the 20th basket)	10	10
A1		
What happens next?		

7. What kind of error is this: syntax, runtime or semantic? Explain.

8. What is the bug? Write the corrected condition.

9. Open the **Variables** panel in Colab while you work. Does it help? What can it not show you?

✓ Before you leave

about 1 minutes

- ☐ I can find what is redundant in a problem and what I really need to track.
- ☐ I can design a normal case and an edge case before coding.
- ☐ I can use `print` to find where a program goes wrong, and name the kind of error.